INSECT SMASHER

PROYECTO VIDEOJUEGOS

Marcos Aranda, Ignacio Villacis y Antonio Muñoz

Explicación del videojuego:

Nuestro videojuego trata sobre aplastar una serie de bichos que salen por pantalla cada uno teniendo su propia velocidad y cantidad de puntos que dar al aplastarlo.

Las reglas son las que siguen, tu objetivo es aplastar la máxima cantidad posible de bichos mientras que un contador en pantalla hace una cuenta atrás, hay varios niveles, cada uno con su fondo individual, para seguir avanzando tendrás que llegar a una cantidad de puntos que saldrá en pantalla, sino se llegase el juego acabaría y te enseñaría una pantalla de “Game Over” donde podrás salir o volver a jugar. Otra manera de perder es si matas 3 veces al gusano rosa, hemos añadido esto para forzar al jugador ha estar atento y que no simplemente aplaste todo lo que sale por pantalla.

En total hay 6 bichos dentro del juego.
La mosca:

La abeja:

El murciélago:

La araña:

La mariquita:

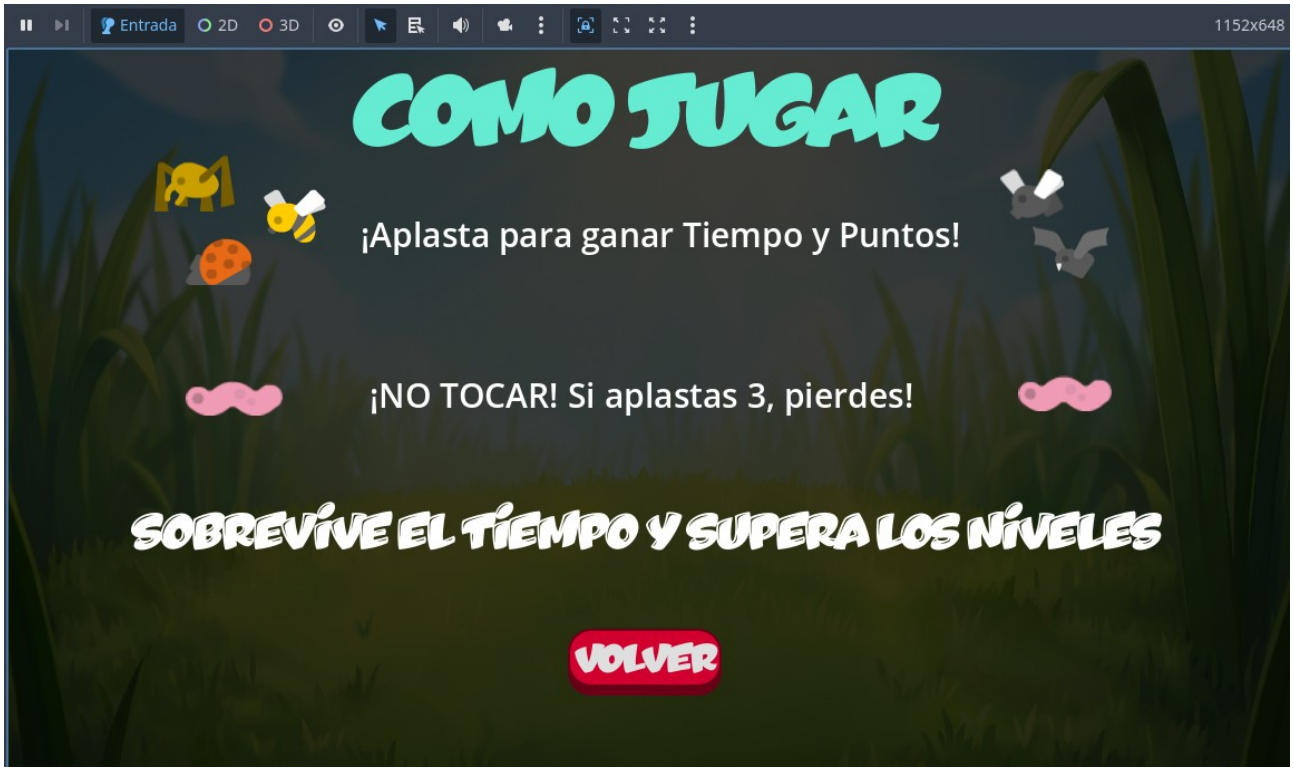
El gusano:

Todos los dibujos de los bichos han sido sacados de Kenney Assets.

Explicación de la parte gráfica del videojuego:

Al iniciar el juego la primera pantalla que te saldrá será la de inicio de juego donde se darán las opciones para jugar, ver las reglas o salir.

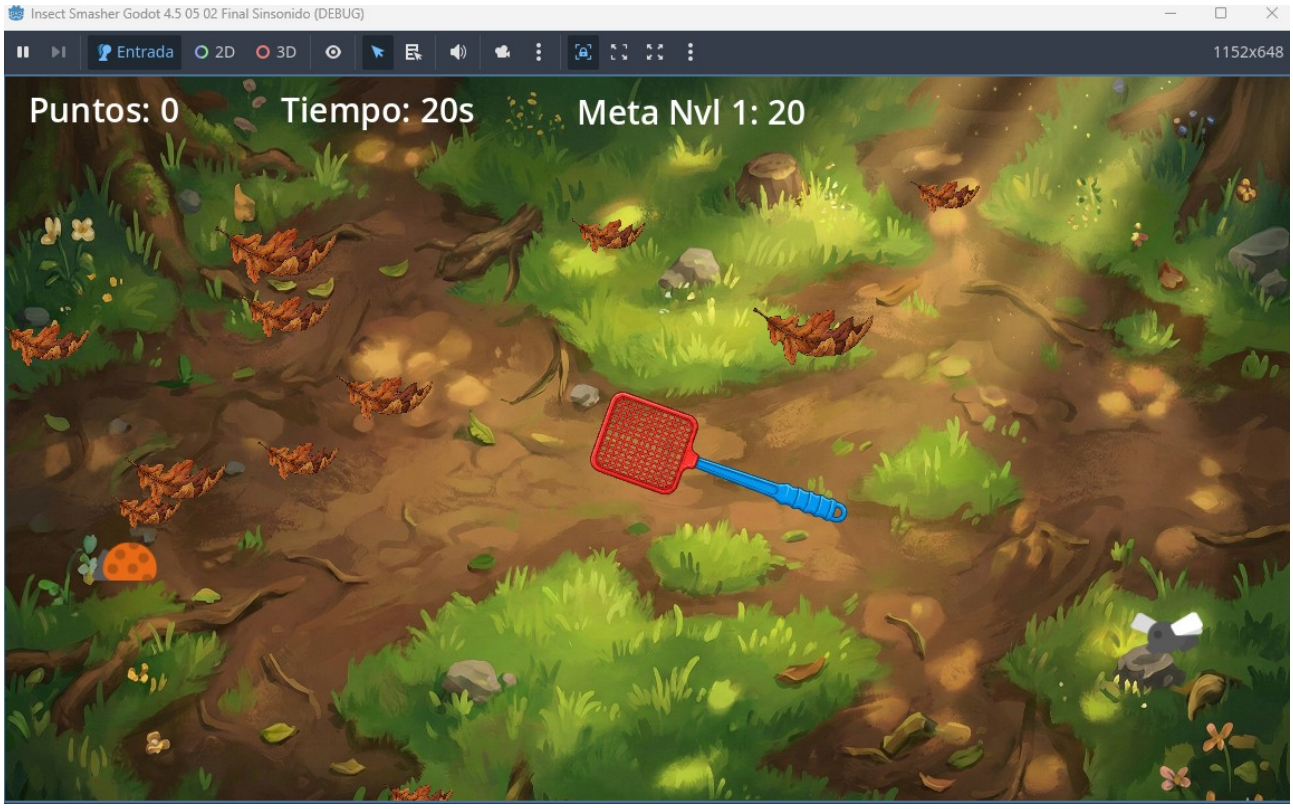
En el apartado de reglas explican brevemente las cosas mas importantes del juego y las cosas con las que tendrás que estar atento.



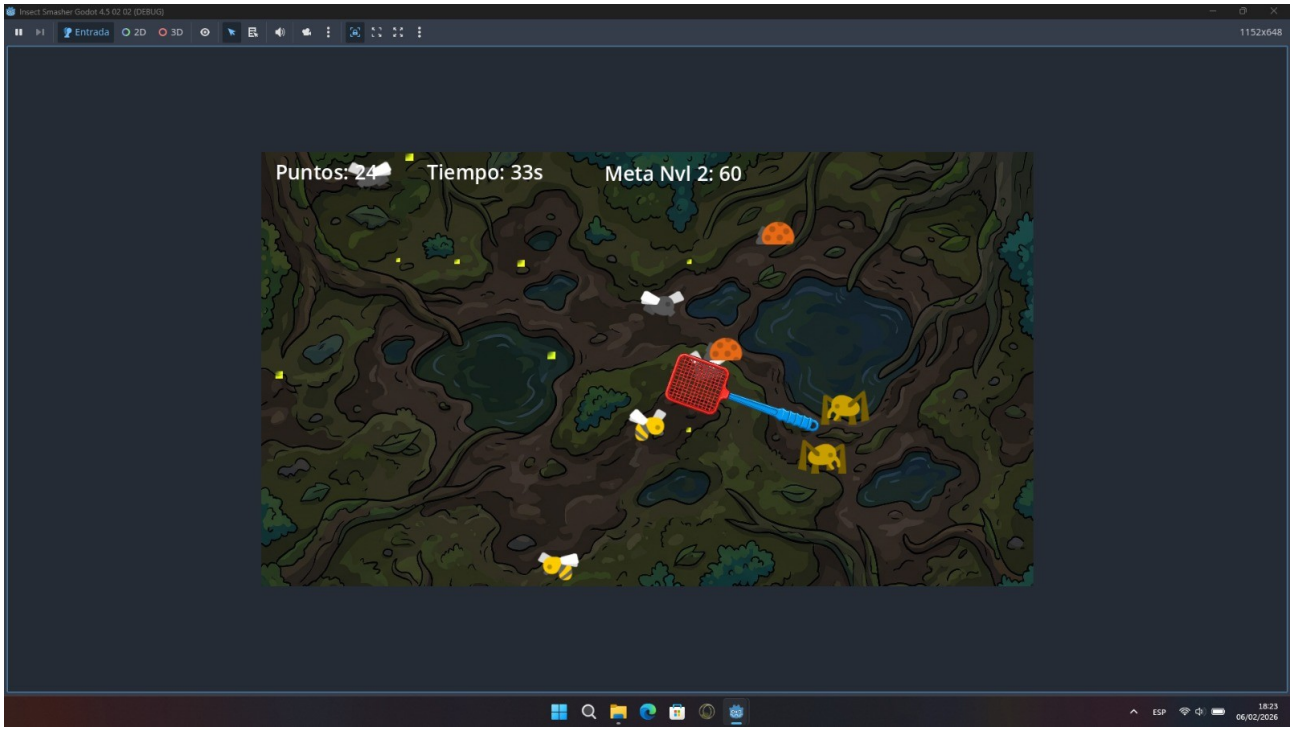
Una vez te sientas preparado para jugar pincha en el botón “Jugar”, una vez le hayas dado ya estarás dentro del juego, donde el jugador maneja el ratón que ha sido sustituido por un matamoscas.

Dentro del juego tenemos distintos fondos para cada nivel:

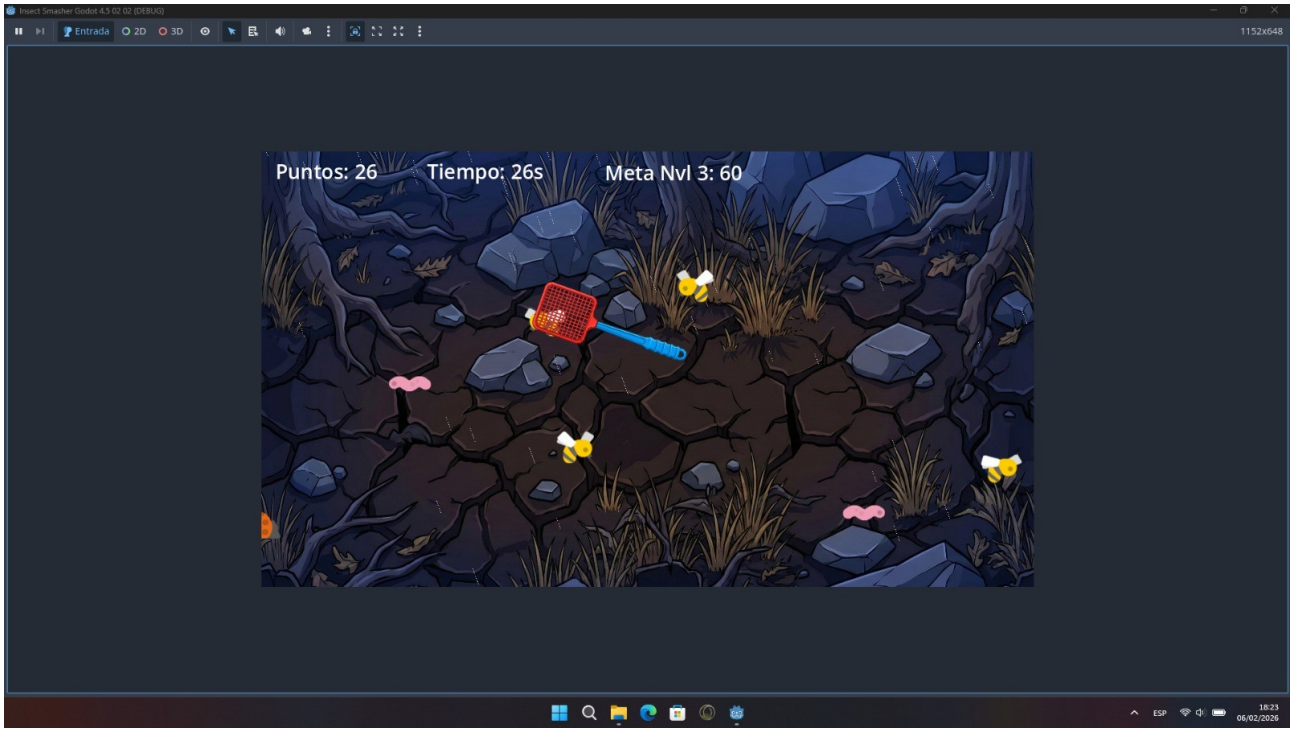
Nivel 1:



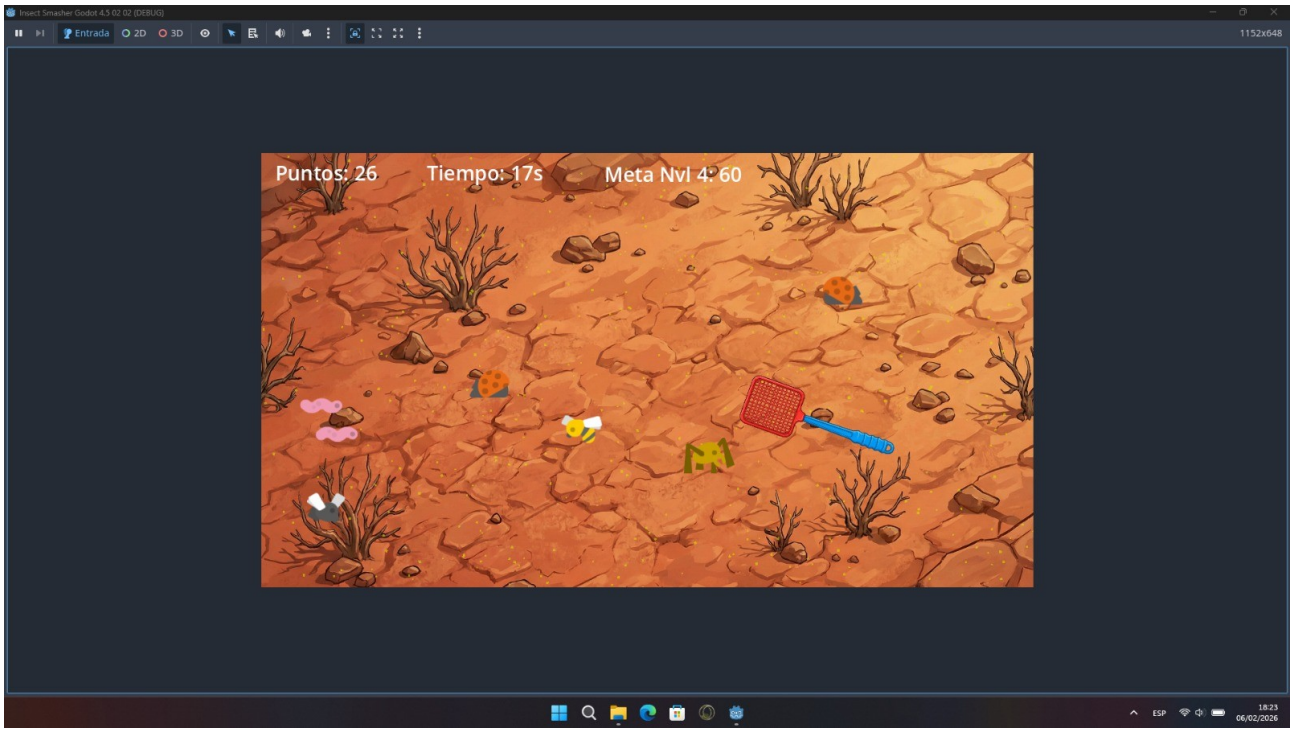
Nivel 2:



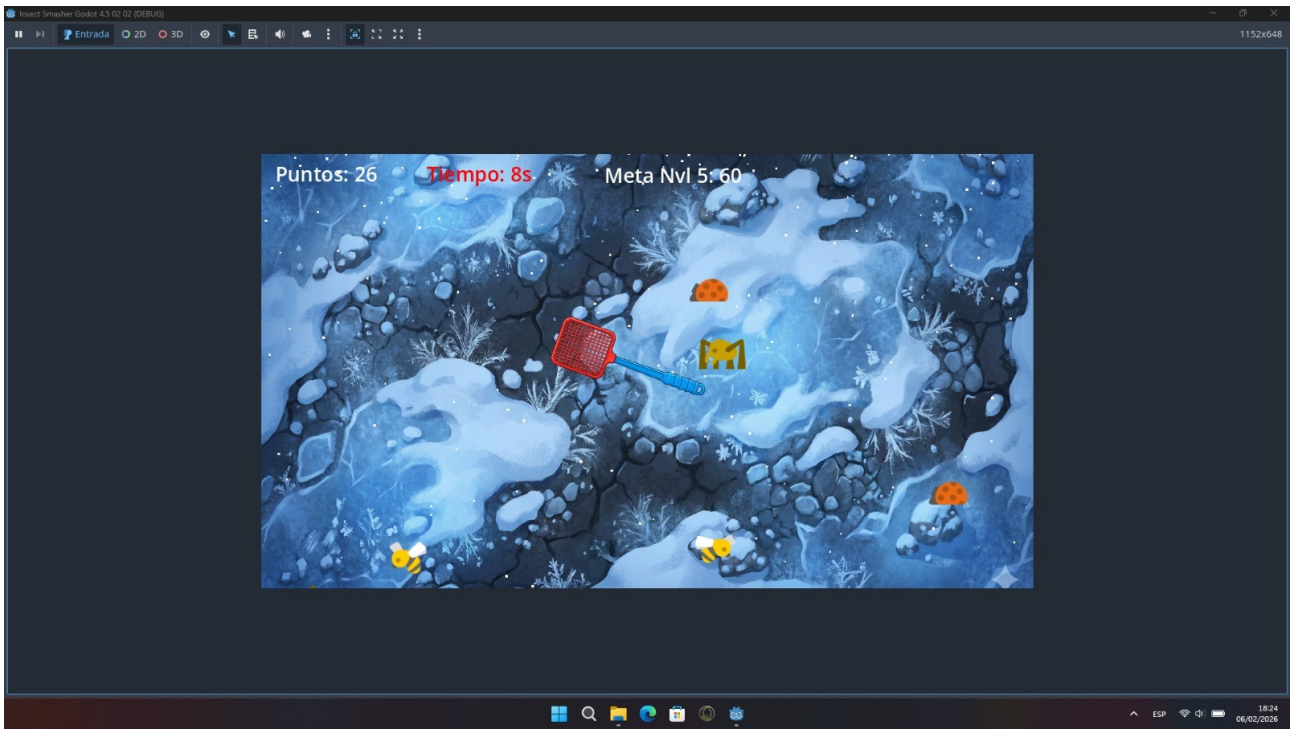
Nivel 3:



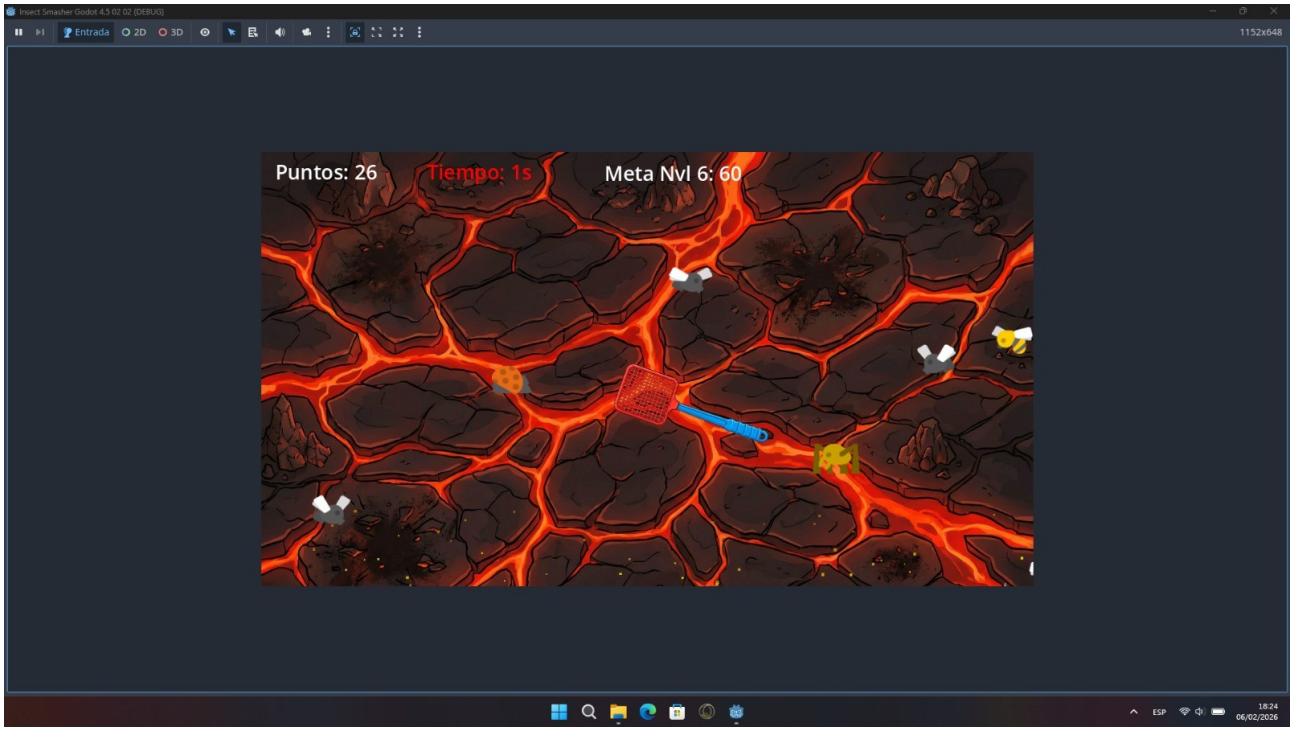
Nivel 4:



Nivel 5:



Nivel 6:



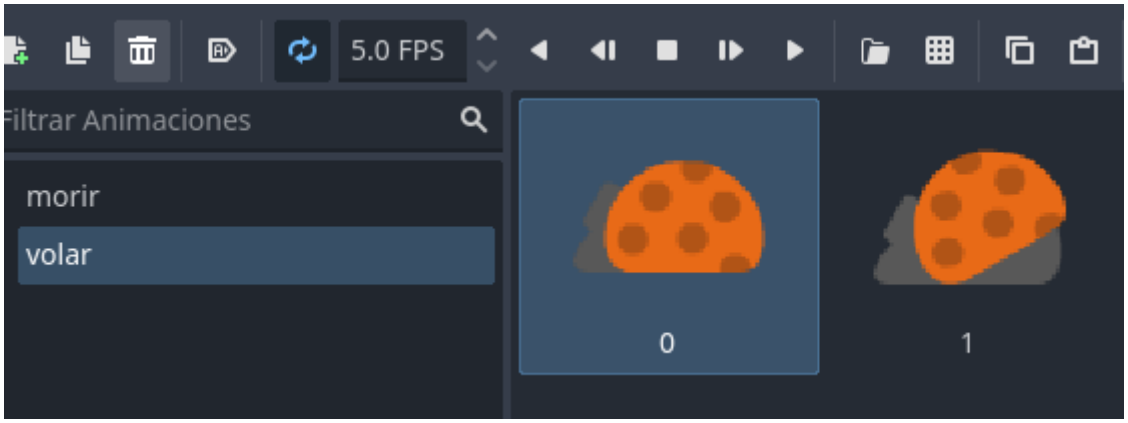
La dificultad va a ir subiendo conforme el tiempo vaya pasando, así que el jugador tiene que tener muy buenos reflejos si quiere tener una buena puntuación.

Explicación de la parte de código del juego:

Para empezar a cada bicho le hemos hecho un .tscn y un .gd prácticamente idénticos solamente cambiando el nombre para las variables.

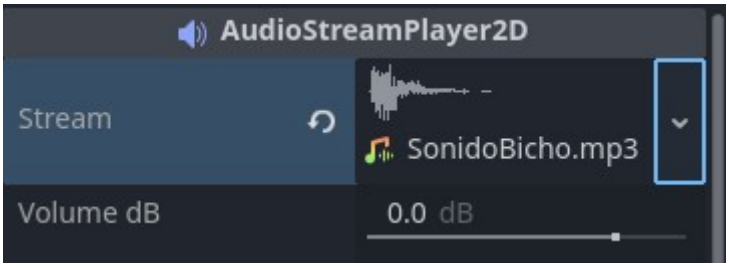
Mariquita.tscn:

Dentro de **“Mariquita.tscn”** hemos añadido un AnimatedSprite2D donde le hemos añadido las animaciones de “volar” y “morir”.



Y también añadimos un CollisionShape2D, para que tengamos algo para pinchar.

Hay tambien un AudioStreamPlayer2D, que incluye “**SonidoBicho.mp3”.**



Y en **“Mariquita.gd”**, que la parte código tenemos:

Primero la declaración de que es un CharacterBody2D, y después declaramos los puntos y la velocidad que queremos que tenga el bicho, y declaramos el estado **“esta_muerto”** como false para que podamos diferenciar para usar las animaciones.

```
extends CharacterBody2D


@export var puntos_que_doy = 5
@export var velocidad = 400.0

var esta_muerto = false
```

Después tenemos el método **“func _ready”** lo cual nos centra la pantalla, declara la velocidad a la que se mueve el bicho y declara que animación usar durante el método, la cual es **“volar”**.
La **“collision_mask”** la hemos dejado en 0 debido que así los bichos no pueden chocarse entre sí, y el **“input_pickable”** lo declaramos como true para que el jugador pueda pinchar al bicho que esta cruzando la pantalla.

```
func _ready():
	var centro_pantalla = get_viewport_rect().size / 2

	var variacion = Vector2(randf_range(-100, 100), randf_range(-100, 100))
	var direccion = ((centro_pantalla + variacion) - position).normalized()

	velocity = direccion * velocidad

	$AnimatedSprite2D.play("volar")

	collision_mask = 0
	input_pickable = true
	add_to_group("bichos")

func _physics_process(delta):
	if esta_muerto: return

	move_and_slide()
```

Y finalmente tenemos el método **“func matar_bicho”** esto convierte el estado **“esta_muerto”** cambia su declaración a true y lo devuelve, cambia la animación a **“morir”**, llama a la clase global, y suma los puntos que vale la mariquita, seguido de un “timer” que espera para borrar después de pincharle.

```
func matar_bicho():
    if esta_muerto: return
    esta_muerto = true

    $CollisionShape2D.set_deferred("disabled", true)
    $AnimatedSprite2D.play("morir")

    # SUMAR PUNTOS
    Global.sumar_puntos(puntos_que_doy)

    # Esperar y borrar
    await get_tree().create_timer(0.5).timeout
    queue_free()
```

En Global.gd es donde se manejan las cantidades de puntos, tiempos, etc. es decir donde se guardan todas las consonantes y variables.

```
extends Node

var Puntos = 0
var Record = 0
var TiempoRestante = 25.0
var Meta = 20
var Nivel = 1


var GusanosGolpeados = 0
var MaxErroresGusano = 3

var MultiplicadorVelocidad = 1.0

# ETIQUETAS
var LabelPuntos
var LabelTiempo
var LabelMeta
```

_ready()

Este es un método incorporado de Godot que se ejecuta una sola vez cuando el nodo entra en la escena.

- **Qué hace:** Configura el process_mode en ALWAYS.
- **Por qué es útil:** Esto asegura que este script siga funcionando incluso si pausas el juego (por ejemplo, para mostrar un menú de pausa).

```
func _ready():
    process_mode = Node.PROCESS_MODE_ALWAYS
```

reset_game()

Esta función se encarga de reiniciar todos los valores a su estado original para empezar una partida nueva.

- **Qué hace:**
    - Pone los puntos y gusanos golpeados a 0.
    - Reinicia el nivel a 1 y la meta a 20.
    - Restaura el tiempo a 25 segundos y la velocidad normal.
    - Al final, llama a **refreshLabels()** para que la pantalla muestre estos nuevos valores inmediatamente.

```
func reset_game():
    Puntos = 0
    Meta = 20
    Nivel = 1
    TiempoRestante = 25.0
    MultiplicadorVelocidad = 1.0
    GusanosGolpeados = 0
    refreshLabels()
```

sumar_puntos(cantidad)

Esta función se llama cada vez que el jugador hace algo bien (probablemente golpear al objetivo correcto).

- **Qué hace:**
    - Suma la cantidad recibida a la variable Puntos.
    - **Verifica nivel:** Si los puntos superan la Meta, llama a la función **subir_nivel()**.

- **Verifica récord:** Si los puntos actuales son mayores que el Record, actualiza el récord.
- Actualiza la interfaz visual.

```
func sumar_puntos(cantidad):
    Puntos += cantidad
    if Puntos >= Meta:
        subir_nivel()
    if Puntos > Record:
        Record = Puntos
    refreshLabels()
```

registrar_golpe_gusano()

Esta función maneja el castigo o error del jugador (golpear lo que no debía).

- **Qué hace:**
    - Aumenta el contador GusanosGolpeados en 1.
    - **Condición de derrota:** Si los errores llegan o superan **MaxErroresGusano** (3), el juego termina y cambia la escena a **"GameOver.tscn"**.

```
func registrar_golpe_gusano():
    GusanosGolpeados += 1
    print("¡ERROR! Gusanos golpeados: " + str(GusanosGolpeados))

    if GusanosGolpeados >= MaxErroresGusano:

        get_tree().change_scene_to_file("res://scenes/GameOver.tscn")
```

subir_nivel()

Se encarga de aumentar la dificultad y dar recompensas cuando el jugador alcanza la meta de puntos.

- **Qué hace:**
    - Sube el número de Nivel.

- **Calcula nueva meta:** Aumenta la meta de puntos necesaria para el siguiente nivel (haciéndolo más difícil progresivamente).
- **Recompensa:** Añade 15 segundos al **“TiempoRestante”**.
- **Dificultad:** Aumenta el MultiplicadorVelocidad en 0.15 (el juego se vuelve más rápido).

```
func subir_nivel():
    Nivel += 1
    Meta += (20 + (Nivel * 10))
    TiempoRestante += 15.0
    MultiplicadorVelocidad += 0.15
    print("¡NIVEL " + str(Nivel) + "! Velocidad x" + str(MultiplicadorVelocidad))
```

refreshLabels()

Esta función actualiza la Interfaz de Usuario (UI) para que el jugador vea los datos actuales.

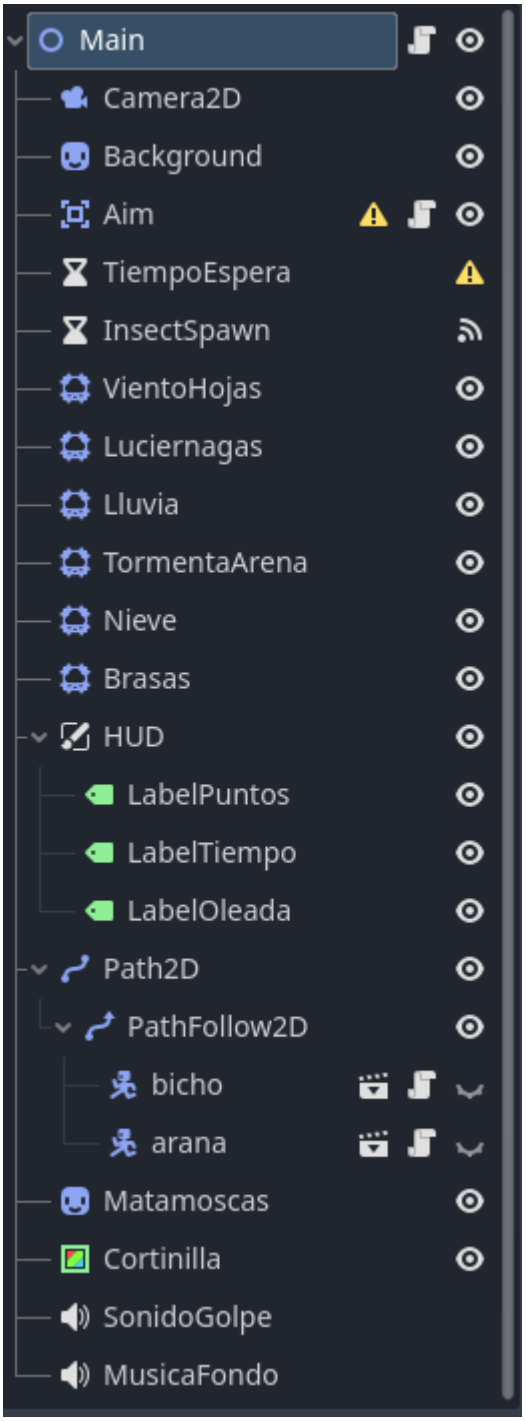
- **Qué hace:**
    - Verifica si las etiquetas (Labels) existen (no son null) para evitar errores.
    - Actualiza el texto de Puntos y Meta.
    - **Lógica visual del tiempo:** Actualiza el texto del tiempo y, si quedan menos de 10 segundos, cambia el color del texto a **Rojo** (Color(1, 0, 0)) para alertar al jugador. Si hay más tiempo, lo mantiene blanco.

```
func refreshLabels():
	if LabelPuntos != null:
		LabelPuntos.text = "Puntos: " + str(Puntos)

	if LabelTiempo != null:
		LabelTiempo.text = "Tiempo: " + str(int(TiempoRestante)) + "s"
		if TiempoRestante < 10:
			LabelTiempo.modulate = Color(1, 0, 0)
		else:
			LabelTiempo.modulate = Color(1, 1, 1)

	if LabelMeta != null:
		LabelMeta.text = "Meta Nvl " + str(Nivel) + ": " + str(Meta)
```

En la escena “Main”, que es la que se usa mientras se juega, está construida de la siguiente manera:



Una “Camera2D” para dejar el plano en 2D, un “Sprite2D” llamado “Background” donde se ha insertado una imagen png para que sea el fondo del nivel.

VientoHojas, Luciernagas, Lluvia, TormentaArena, Nieve y Brasas son todos “CPUParticles2D”, que simulan el fondo como uno animado.

Dentro del HUD hemos añadido tres **“Labels”**: **LabelPuntos, LabelTiempo y LabelOleada**. Que marcan la cantidad de puntos, el tiempo que queda y el nivel en el que estas respectivamente.

Despues hemos creado un **“Path2D”** que como se vio en el ejemplo **“Dodge the creeps”**, sirve para marcar la arena en la que queremos que aparezcan los enemigos, le añadimos el nodo hijo **“PathFollow2D”** para marcar el camino por el que tienen que pasar los enemigos, añadiendo dentro como nodos hijos a los bichos que tienen que seguir este reglamento.

Añadimos como un **“Sprite2D”** el png del matamoscas con el que vamos a aplastar a los bichos.

Y los **“AudioStreamPlayer”** de **MusicaFondo** y **SonidoGolpe**, para añadirle detalles al juego.